

Information Disclosure Walkthrough and Solution

Target from supplied challenge panel: /play/KF42BRI. This report documents the intended flag-finding path and a safe remediation.

Submission summary

Flag	Exact team flag cannot be truthfully printed from the supplied screenshot and source alone. The application creates this value randomly for each team. Retrieve and paste the value shown at /play/KF42BRI/staff-notes: FLAG{info_disclosure_...}.
Fix Suggestion (Optional)	Remove the staff-notes route from the deployed application. Do not place credentials or flags in client-reachable pages, comments, source maps, or build artifacts. If an internal page must exist, protect it with server-side authorization and return 404 to unauthorized requests.

Important flag note. The provided evidence exposes the route and flag format, but not the random database value for team KF42BRI. The value is generated with cryptographic randomness when the team is provisioned; substituting a guessed suffix would be incorrect. A live QuickCart tab or the challenge URL is needed for the final copy step.

What the finding establishes

The storefront ships an internal operational note in the home page HTML. That note reveals a directly reachable staff endpoint. The endpoint checks only that the team code exists and then renders the information-disclosure flag; it does not require an authenticated staff session. A visitor who can open the storefront can therefore follow the disclosed path and read the secret.

Evidence chain

The source of the QuickCart home page constructs this HTML comment:

```
<!-- TODO: remove test note before launch. Debug path: /play/KF42BRI/staff-notes -->
```

Step-by-step flag-finding process

Use these steps against the authorized QuickCart challenge instance. They are written for the team code visible in the supplied panel; preserve the same code throughout.

The referenced page loads the matching team and prints the information-disclosure flag. No staff authorization condition exists between that lookup and rendering the flag.

1. Open the challenge storefront. Select “Open QuickCart” in the challenge panel. The target should open at `/play/KF42BRI`. The visible page looks like an ordinary training storefront and does not display the flag in its rendered content.
2. Inspect the response rather than only the rendered interface. Use the browser’s page-source view or developer tools and search the initial document for words such as `TODO`, `debug`, `staff`, or `note`. This matters because HTML comments do not appear in the normal page view but remain available in the delivered markup.
3. Identify the leaked internal path. The homepage contains a comment that names the debug route `/play/KF42BRI/staff-notes`. Treat that as a routing clue, not as an instruction to authenticate: it is publicly reachable for the existing team code.
4. Request the disclosed route. Replace the browser path with `/play/KF42BRI/staff-notes` and load it. The route returns an “Internal staff note” page with a pre-launch checklist.
5. Copy the complete flag. Under “Debug credential issued for this storefront instance,” copy the entire literal value beginning with `FLAG{` and ending with `}`. Do not add whitespace or alter the random suffix.
6. Submit it to the challenge form. Return to the Information Disclosure panel, paste the copied value into Flag, and provide the requested structured report in the application. The supplied challenge UI may require its own mandatory fields even though this document intentionally keeps its summary limited to the fields requested for this deliverable.

Expected successful observation

The staff-notes page should visibly identify itself as an internal note and show one team-specific credential in the `FLAG{info_disclosure_...}` format. If the page is missing, verify the team code from the challenge URL and ensure the event environment is currently available.

Technical explanation and remediation

Why the route exposes the flag

The home route deliberately emits an HTML comment containing a debug path. Comments are not a security boundary: browsers receive them as part of the response, and users can inspect them. The staff-notes route then retrieves the flag associated with the team code and renders it. The only gate is whether the code maps to a valid team; no staff-role or session check protects the route.

Correct remediation pattern

The durable fix is to remove debug and staff-only endpoints from participant deployments. If operational content must be retained, enforce authorization on the server before accessing or rendering the sensitive value. Keep all secrets out of HTML, JavaScript bundles, client-side route definitions, comments, logs, and source maps. Finally, rotate any exposed credential and add a regression test that confirms unauthorized requests to internal routes receive a 404 or 403 without revealing a secret.

Verification checklist for the fix

- Open the storefront source and confirm it contains no debug path, internal note, or credential reference.
- Request the former staff-notes URL as an unauthenticated participant and confirm it returns no sensitive content.
- Search the production build output and source maps for the old route and for values matching the flag format.
- Confirm a newly issued secret replaces the exposed one before the environment is reused.

Evidence basis

This walkthrough was derived from the supplied QuickCart challenge panel and the associated application implementation. The exact flag value is intentionally not fabricated: the implementation generates a different random value for each provisioned team. Add the live value from the staff-notes page to the Flag field before final challenge submission.